Benchmarking a Basic Triple-Loop Matrix Multiplication in Python, Java, C and Go

Aythami Lorenzo

October 2026 — Individual Assignment 1, Big Data

Abstract

We study the behavior of several basic matrix multiplication algorithms used for multiplying dense square matrices in four programming languages: Python, Java, C and Go. All implementations execute the same naive *i-j-k* triple loop over `float64` values with row-major storage, so the four implementations differ only in language and runtime. We perform an analysis of the following elements: kernel-only runtime per multiplication (median and interquartile range over 7 repetitions after 3 warm-ups), peak resident set size of the process, and result checksums validated against a NumPy reference. Our experiments provide a reproducible benchmark of $O(n^3)$ algorithms across a variety of input sizes ($n = 10$ to 1000 elements per side) to guide future research in the cost of language runtimes for naive numeric kernels. Based on our experiments, we recommend C when raw throughput is the goal, Go as a portable near-C alternative with a simple memory profile, Java once its just-in-time warm-up is accounted for, and Python only for small matrices or prototyping, where its 50–90× slowdown remains affordable.

Contents

1 Introduction and objective **1**

2 Background **2**

3 Methodology **2**
3.1 Implementations and protocol 2

4 Results **3**
4.1 Runtime 3
4.2 Memory 3

5 Discussion **3**

6 Conclusions **4**

7 References **5**

8 Reproducibility materials **6**

1 Introduction and objective

Dense matrix multiplication is the canonical workload for studying how a programming language and its runtime affect the cost of a purely arithmetic $O(n^3)$ kernel. Because the algorithm and its memory-access pattern—a naive triple loop with a strided column scan—are identical across implementations, any observed difference in runtime or memory can be attributed to the language, its compiler or just-in-time (JIT) engine, and its runtime system.

The objective of this assignment is to answer the research question: *how does the matrix size n affect the runtime and the peak memory of a basic $O(n^3)$ triple-loop multiplication across four*

implementations? The assignment requires Python, Java and C; we add Go as a clearly labelled fourth implementation that follows the identical algorithm and protocol, so it is compared but never substitutes the required three-language comparison.

2 Background

The textbook algorithm computes each entry of $C = AB$ as a dot product of one row of A with one column of B, giving n^3 multiply–accumulate operations, i.e. $\Theta(n^3)$ arithmetic work [1]. Gauss already observed that the operation count can be reduced asymptotically [2], but the basic triple loop remains the reference for clarity and the subject of this study.

Arithmetic is only part of the cost: the inner loop streams one row of A sequentially while jumping through B with a stride of $8n$ bytes, so once the working set (three matrices, $24n^2$ bytes) exceeds the last-level cache, each iteration becomes a memory transaction [4, 3]. All values are IEEE-754 `float64` numbers [6], so results are deterministic for a fixed evaluation order and comparable across languages up to the last unit in the last place (ulp).

3 Methodology

3.1 Implementations and protocol

All four kernels perform the same loop nest (i, then j, then k) writing into a pre-allocated `float64` row-major output buffer; no library call (BLAS, `numpy.matmul`, …) executes inside the measured kernel. NumPy is used only to generate the shared inputs [5] and to compute the correctness reference; the measured Python kernel uses plain lists of lists of `float`.

- **Shared inputs.** Every language loads the same `matrixA_n.bin` / `matrixB_n.bin` files (little-endian `float64`, row-major), generated once with seed 42; generation and loading happen outside the timed region.
- **Timing boundary.** The timer surrounds only the triple loop; a sequential row-major checksum is computed afterwards so the compiler cannot eliminate the work. Only monotonic clocks: `System.nanoTime()` (Java), `clock_gettime(CLOCK_MONOTONIC)` (C), `time.Now()` (Go), `time.perf_counter_ns()` (Python).
- **Warm-up and repetitions.** Three warm-up runs (recorded, excluded) plus seven measured repetitions; we report the median and the interquartile range (IQR). Multiplications shorter than ≈2 ms run as a batch of K multiplications inside the timed region, reporting time per multiplication (K stored in the CSV).
- **Budgets and sizes.** 60 s per multiplication, ≈2 GB memory; a budget overrun is a result (exit code 3), never an invented time. Sizes: $n \in \{10, 25, 50, 100, 200, 500, 1000\}$.
- **Memory metric.** Peak resident set size (peak RSS) read from the kernel watermark `/proc/self/status` (`VmHWM`) in *all four* languages—one metric, one counter, one tool. The theoretical array storage $24n^2$ and, for Java, the peak heap are reported separately and never compared as if identical.
- **Correctness.** Before any benchmark, each implementation is validated against a NumPy `float64` reference with `rtol`=1e-9, `atol`=1e-12 on identity, zero, known-answer 3×3 and random small matrices. On the campaign inputs the four checksums agree exactly for $n \leq 500$ and within 1 ulp at $n = 1000$.

All builds, tests, benchmarks and plots ran inside WSL2 on the student's machine (Table 1); native Windows binaries are blocked by Smart App Control on this computer, so only measurements taken in this environment are reported. Each configuration ran sequentially, once, on an otherwise idle machine.

Table 1: Execution environment of the campaign (WSL2 on the student's machine).

Component	Value
CPU	Intel Core i7-10750H @ 2.6 GHz (6 cores / 12 threads)
Host memory / OS	16 GB DDR4, Windows 11 host
Execution OS	WSL2, Ubuntu 26.04 LTS, kernel 6.18.33.2
VM memory / CPUs	≈7.7 GB RAM, 12 logical processors
C compiler	gcc 15.2.0, `-O2 -std=c11 -Wall -Wextra`
Java	OpenJDK 17.0.20.1, Maven 3.9.12
Go / Python	Go 1.27.2 / Python 3.14.4 (numpy, pandas, matplotlib)

Table 2: Median time per multiplication in ms (IQR in parentheses), 7 repetitions after 3 warm-ups. "–": budget exceeded, no measurement.

n	C	Go	Java	Python
10	0.00086 (0.00003)	0.0027 (0.00011)	0.058 (0.016)	0.078 (0.003)
25	0.016 (0.0003)	0.056 (0.002)	0.16 (0.030)	1.12 (0.043)
50	0.158 (0.012)	0.421 (0.010)	1.26 (0.14)	8.73 (0.11)
100	1.47 (0.04)	3.31 (0.11)	1.95 (0.16)	75.3 (4.8)
200	13.8 (0.3)	25.6 (0.5)	17.0 (0.6)	672 (6.2)
500	238 (1.7)	405 (13)	350 (7.6)	13 711 (146)
1000	5 672 (88)	6 414 (58)	17 539 (68)	–

4 Results

4.1 Runtime

Table 2 reports the median time per multiplication with the IQR of the seven measured repetitions; Figure 1 shows the same data on log–log axes with the $O(n^3)$ reference slope.

Figure 2 (left) shows the slowdown relative to C: C is fastest at every size; Go runs 1.1–3.5× slower, Python 50–90× slower, and Java moves from 67× slower at $n = 10$ to 1.2–1.5× in the mid range and 3.1× at $n = 1000$. All curves follow the $O(n^3)$ trend over three orders of magnitude, with a steeper segment between $n = 500$ and $n = 1000$ discussed in Section 5. Right: Figure 2 (right) reports the IQR as a percentage of the median: C, Go and Python stay mostly below 5 % once the timed region is long enough, while Java shows 8–28 % variability for $n \leq 100$—consistent with JIT compilation still progressing during the measurement window—and drops below 4 % for $n \geq 200$.

4.2 Memory

Table 3 and Figure 3 show two regimes: a flat runtime floor at small n (C: 2.1 MB; Go: 2.6 MB; Python: 16.9 MB; JVM: 46.2 MB) dominated by the interpreter, loader and runtime structures, and a growth regime where the $24n^2$ bytes of matrix data dominate. At $n = 1000$, C peaks at 26.1 MB—the theoretical 24 MB plus ≈2 MB of process overhead—while the JVM peaks at 117.4 MB, of which only ≈3 MB is heap.

5 Discussion

Complexity and the cache cliff. The measured curves track the theoretical $O(n^3)$ law over three orders of magnitude, but the growth from $n = 500$ to $n = 1000$ is steeper than the pure arithmetic prediction (×8): ×23.9 for C, ×15.8 for Go and ×50.1 for Java. The cause is the memory hierarchy: at $n = 1000$ the three matrices occupy 24 MB, well above the 12 MB last-level cache of the i7-10750H, so the strided column reads of B become cache misses [3, 4]. This is a property of the algorithm and the machine, not of any language.

Language effects. C is fastest at all sizes. Go stays within 1.1–3.5× of C—its ahead-of-time compiled kernel runs without a garbage collector inside the loop—at the price of a slightly higher RSS (35.5 MB vs 26.1 MB at $n = 1000$). Java converges toward C in the mid range once the JIT compiles

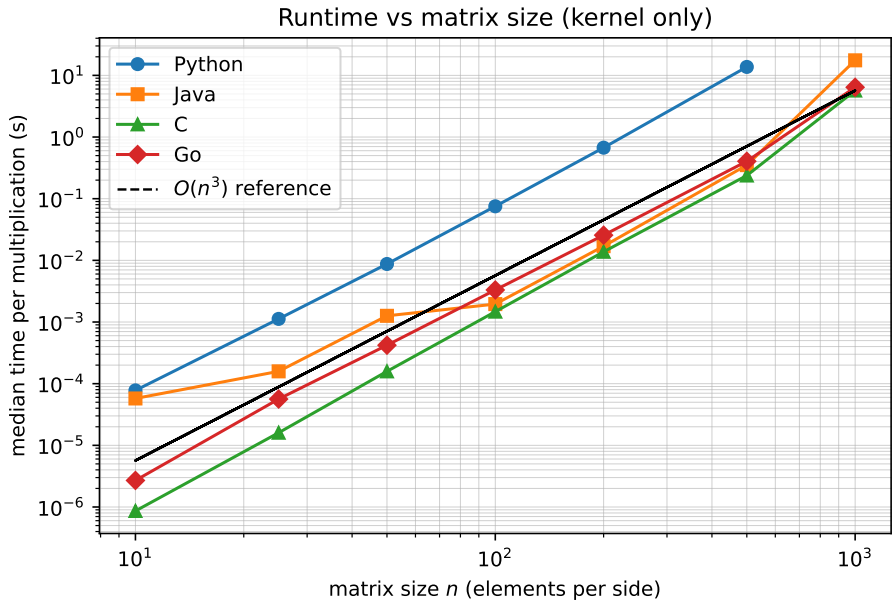


Figure 1: Median kernel runtime vs matrix size (log–log) with the $O(n^3)$ reference slope. Python has no point at $n = 1000$: the run exceeded the 60 s budget.

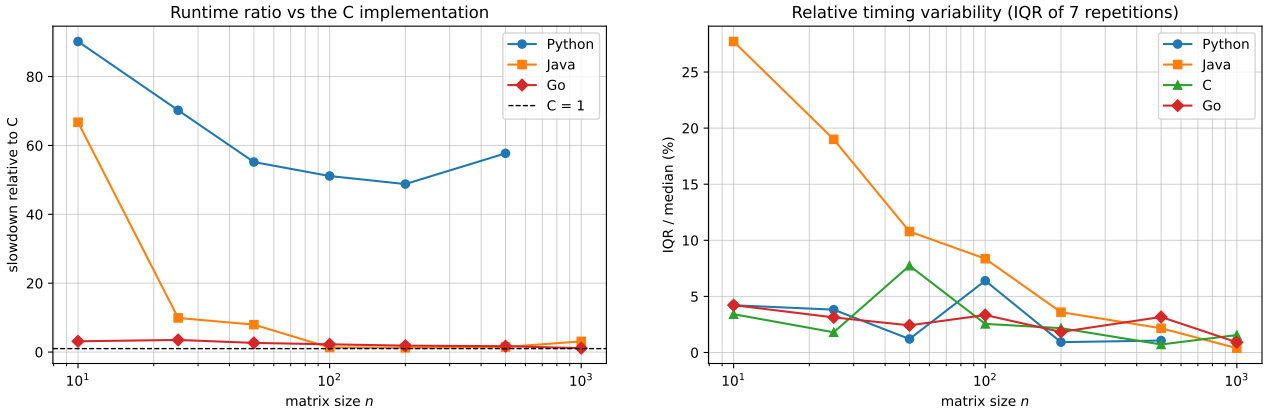


Figure 2: (Left) Runtime ratio of each language relative to C. (Right) Relative timing variability (IQR/median over the 7 measured repetitions).

the hot loop, then loses ground again at $n = 1000$; its higher small-size variability reflects the same warm-up effect. Python is 50–90× slower throughout; its flat interpreter overhead also dominates at small n, where the reported times are those of an interpreter loop rather than of arithmetic.

Budget outcome. Python at $n = 1000$ required 178.6 s for a single probe multiplication, exceeding the 60 s budget; the runner exited with code 3 and recorded no time or memory row for that configuration. The largest completed size therefore differs per language (Python: $n = 500$; the other three: $n = 1000$), which is itself a protocol result and is reported as such.

6 Conclusions

The basic i-j-k triple-loop multiplication behaves as $O(n^3)$ predicts up to the point where the working set leaves the last-level cache, at which point memory traffic steepens the curve for every language alike. Language choice changes the constant factor, not the complexity: C is fastest; Go provides near-C speed with a clean, portable build; Java is competitive after JIT warm-up but needs it accounted for; Python is one to two orders of magnitude slower and hits the 60 s budget first. Peak RSS grows

Table 3: Peak RSS in MB (from `VmHWM`) and theoretical array storage $24n^2$. "–": not recorded (budget exceeded).

n	$24n^2$	C	Go	Java	Python
10	0.0	2.1	2.6	46.2	16.9
50	0.1	2.1	2.8	46.4	17.2
100	0.2	2.3	3.0	46.9	18.0
200	1.0	3.0	4.3	48.4	21.8
500	6.0	8.1	13.3	59.0	48.7
1000	24.0	26.1	35.5	117.4	–

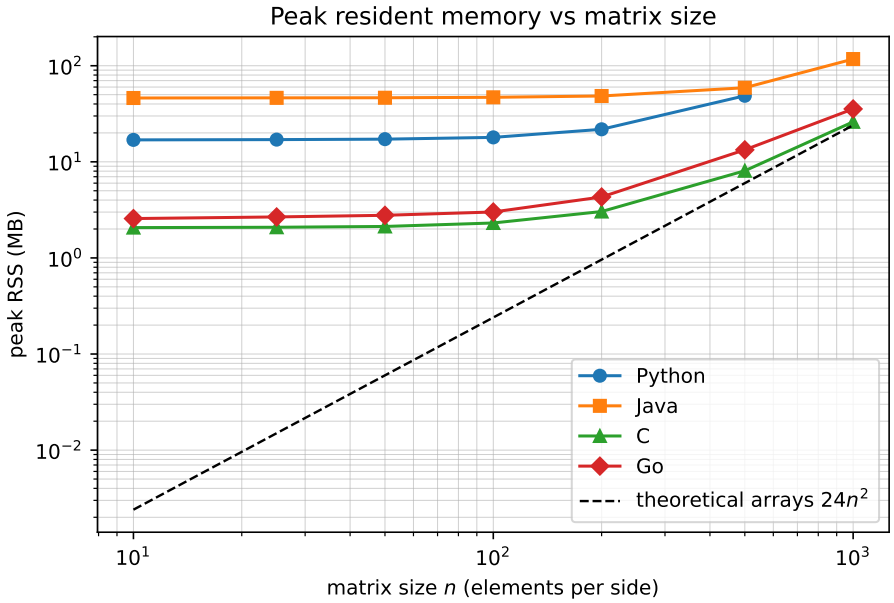


Figure 3: Peak resident set size vs matrix size, with the theoretical array storage $24n^2$ as reference.

toward the theoretical $24n^2$ bytes only when the matrices dominate the runtime's own memory floor.

Recommendations. Use C for maximum throughput on this workload; Go when portability and near-C performance matter; Java with explicit warm-up excluded from the measured window; and Python for sizes where its slowdown is acceptable ($n \lesssim 200$). Always report the warm-up policy, the batch factor K, and budget outcomes together with the medians.

7 References

References

[1] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.

[2] V. Strassen, "Gaussian elimination is not optimal," *Numerische Mathematik*, vol. 13, no. 4, pp. 274–279, 1969.

[3] M. Frigo, C. E. Leiserson, H. Prokop, and S. Ramachandran, "Cache-oblivious algorithms," in *Proc. 40th IEEE Symp. Foundations of Computer Science (FOCS)*, 1999, pp. 285–297.

[4] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. San Mateo, CA, USA: Morgan Kaufmann, 2017.

[5] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[6] IEEE Std 754-2019, *IEEE Standard for Floating-Point Arithmetic*, IEEE, 2019.

[7] T. Lindholm, J. Yellin, G. Bracha, and A. Buckley, *The Java Virtual Machine Specification, Java SE 17 Edition*. Oracle, 2021.

[8] A. Donovan and B. W. Kernighan, *The Go Programming Language*. Boston, MA, USA: Addison-Wesley, 2015.

[9] Python Software Foundation, "`time` — Time access and conversions," *Python 3.14 Documentation*, 2026. [Online]. Available: https://docs.python.org/3/library/time.html

[10] Microsoft, "What is WSL?," *Microsoft Learn*, 2026. [Online]. Available: https://learn.microsoft.com/windows/wsl/

8 Reproducibility materials

All materials ship with this report: the Java (Maven), C, Go and Python sources; the JUnit, C, Go and Python correctness tests; the shared input generator; the raw campaign CSVs (`data/results/time_raw.csv`, `memory_raw.csv`, `run_log.txt`); and the figure script plus figure PDFs. From the project root, inside WSL2:

```
.venv/bin/python python/generate_inputs.py
mvn -q verify
bash c/build.sh && ./c/correctness_test data/reference
cd go && gofmt -l . && go vet ./... && go test ./... && cd ..
.venv/bin/python python/correctness_check.py \
    --reference-dir data/reference
.venv/bin/python python/run_experiments.py    # ~12 min campaign
.venv/bin/python python/plot_results.py       # -> figures/*.pdf
```

Exact toolchain versions and the full protocol are documented in `README.md`. Runner exit codes: 0 (ok), 2 (invalid arguments), 3 (time budget exceeded), 4 (missing input files).